CPPDescent

8762ab8 (with uncommitted changes)

Generated by Doxygen 1.9.7

Chapter 1

CPP-Descent

Part of the "Software Development for Computing Systems" course of the Department of Informatics and Telecommunications, UoA for the first semester of the academic year 2023-2024.

A library that implements the KNN-Graph creation and search algorithms described in [1], [2].

1.1 Contributors

- Konstantinos Chousos (1115202000215, sdi2000215 *at* di.uoa.gr)
  - 1st submission
    - Build/Testing/Code coverage pipeline
    - ADTVector, ADTList, ADTPQueue
    - Brute Force K-NN Graph creation. NN-Descent Graph creation
    - (Incomplete) Query point K-NN
  - 2nd submission
    - I/O of graphs in binary files
    - Recall function
    - Metric functions
    - Early termination
- Anastasios-Fedon Seitanidis (1115202000179, sdi2000179 *at* di.uoa.gr)
  - 1st submission
    - ADTMap, ADTGraph
    - Brute Force K-NN Graph creation
    - NN-Descent Graph creation

1.2 Dependencies

For local development, you will need `Make` and `CMake`. To install them on an apt-based linux distro, run the following commands:

```
$ sudo apt update && sudo apt upgrade
$ sudo apt install make
$ sudo apt install cmake
```

1.2.1 Minor dependencies

- The project uses Google Test for its testing needs. Gtest is used as a git submodule of this repo, that is downloaded by CMake the first time you clone. That means that nothing extra needs to be installed to run the tests.
- If you would like to generate the documentation, you will need `doxygen` installed.

1.3 Running the project

To run the main executable, the following commands suffice:

```
$ cmake -S . -B build
$ cmake --build build
$ ./build/app/app <K> <filepath-to-dataset> <dimensions> <1 or 2>
```

The last argument concerns the metric function that will be used. 1 means that the euclidean distance will be used and 2 stands for the manhattan distance.

1.3.1 Results

The *recall* percentage that is printed during execution is computed by the following formula, derived from [1, Sec. 3.2].

> The recall of one object is the number of its true K-NN members found divided by K. The recall of an approximate K-NNG is the average recall of all objects.

1.4 Code Structure

In the wiki, you can find documentation generated by Doxygen. It is also available in pdf form.

1.4.1 Code Coverage

This project uses LCOV for its code coverage needs. The results are uploaded upon commit here.

1.5 Bibliography

[1] W. Dong, C. Moses, and K. Li, “Efficient k-nearest neighbor graph construction for generic similarity measures,” in Proceedings of the 20th international conference on World wide web, in WWW ’11. New York, NY, USA: Association for Computing Machinery, Mar. 2011, pp. 577–586. doi: 10.1145/1963405.1963487.

[2] “How PyNNDescent works — pynndescent 0.5.0 documentation.” Accessed: Oct. 10, 2023. [Online]. Available: https://pynndescent.readthedocs.io/en/latest/how_pynndescent_works.html

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

Chapter 5

Namespace Documentation

5.1 cppdescent Namespace Reference

Functions for the creation of a K-NN graph.

Functions

- void **deleteFloat** (Pointer value)
- float * **createFloat** (float value)
- float **compareFloats** (Pointer a, Pointer b)
- int **deleteDatapointVectors** (Vector *vec)
- int **compareVertices** (Pointer first, Pointer second)
- Vector * readBinData (const char *fp, int dimensions)

  Reads the data from a binary file.
- void writeBinGraph (const char *fp, Graph *graph, int K)

  Writes a computed graph to a binary file.
- Graph * readBinGraph (const char *fp, int dimensions, DistanceFunc distance)

  Reads a graph from a binary file. To work correctly, the file needs to be first created from the 'writeBinGraph' function.
- float recall (Graph *bfGraph, Graph *nnGraph, int N, int K)

  Returns the recall of the graph computed by NN-Descent, compared to the brute force graph.
- float euclideanDistance (Pointer a, Pointer b)

  Returns the Euclidean distance between two points of arbitrary dimension.
- float manhattanDistance (Pointer a, Pointer b)

  Returns the Manhattan distance between two points of arbitrary dimension.
- int compareEdgesEuclidean (Pointer first, Pointer second)

  Compare edges using the euclideanDistance function.
- int compareEdgesManhattan (Pointer first, Pointer second)

  Compare edges using the manhattanDistance function.
- Graph * KNNBruteForceGraph (Vector *data, int K, CompareFunc compare, DistanceFunc distance)

  Computes the K-NN graph using brute force.
- Graph * NNDescent_KNNGraph (Vector *data, int K, float delta, DistanceFunc distance)

  Computes the K-NN graph for the given dataset using the NN-Descent algorithm.
- List * NNDescent_Query (Graph *graph, int K, CompareFunc compare, Vector *query)

  Computes the K Nearest Neighbors of the query point in the graph.

5.1.1 Detailed Description

Functions for the creation of a K-NN graph.

5.1.2 Function Documentation

5.1.2.1 compareEdgesEuclidean()

```
int cppdescent::compareEdgesEuclidean (
            Pointer first,
            Pointer second )
```

Compare edges using the euclideanDistance function.

Parameters

first	A Pointer to the first element.
second	A Pointer to the second element.

Returns

int

5.1.2.2 compareEdgesManhattan()

```
int cppdescent::compareEdgesManhattan (
            Pointer first,
            Pointer second )
```

Compare edges using the manhattanDistance function.

Parameters

first	A Pointer to the first element.
second	A Pointer to the second element.

Returns

int

5.1.2.3 euclideanDistance()

```
float cppdescent::euclideanDistance (
            Pointer a,
            Pointer b )
```

Returns the Euclidean distance between two points of arbitrary dimension.

Parameters

first	A pointer to the first point.
second	A pointer to the second point.

Returns

long double The Euclidean distance.

5.1.2.4 KNNBruteForceGraph()

```
Graph * cppdescent::KNNBruteForceGraph (
            Vector * data,
            int K,
            CompareFunc compare,
            DistanceFunc distance )
```

Computes the K-NN graph using brute force.

Useful for testing the recall and the performance of our NN-Descent algorithm.

The user is responsible deletion of the allocated memory of the graph.

Parameters

data	A pointer to the parent N-sized vector.
K	The number of Nearest Neigbors to find.
compare	The function to use to compare the distances.
distance	The function that computes the distance between two points (vectors).

Returns

Graph∗ A pointer to the optimal K-NN graph.

5.1.2.5 manhattanDistance()

```
float cppdescent::manhattanDistance (
            Pointer a,
            Pointer b )
```

Returns the Manhattan distance between two points of arbitrary dimension.

Parameters

first	A pointer to the first point.
second	A pointer to the second point.

Returns

long double The Manhattan distance.

5.1.2.6 NNDescent_KNNGraph()

```
Graph * cppdescent::NNDescent_KNNGraph (
            Vector * data,
            int K,
            float delta,
            DistanceFunc distance )
```

Computes the K-NN graph for the given dataset using the NN-Descent algorithm.

The user is responsible deletion of the allocated memory of the returned graph.

Parameters

data	A Vector of the vertices of the graph.
K	The number of nearest neighbors to compute.
delta	The iterations will stop when the number of edges that were updated is less than delta∗N∗K.
distance	The function to be used to compute the distances between vertices.

Returns

Graph∗ The complete K-NN graph of the dataset.

5.1.2.7 NNDescent_Query()

```
List * cppdescent::NNDescent_Query (
            Graph * graph,
            int K,
            CompareFunc compare,
            Vector * query )
```

Computes the K Nearest Neighbors of the query point in the graph.

Parameters

graph	The K-NN Graph, in which we search for neighbors of the query.
K	
compare	The function used to compare edges.
query	The query point, given as a Vector. Must be of the same dimensions as the rest points of the graph.

Returns

List∗ A list of the K-NN of the query. If the query point is of wrong dimensions, then nullptr is returned.

5.1.2.8 readBinData()

```
Vector * cppdescent::readBinData (
            const char * fp,
            int dimensions )
```

Reads the data from a binary file.

The data read are stored in a N-sized Vector, where N is the <uint32_t> at the start of the file. Each node of this vector is a pointer to another 100-sized vector, of which the values are pointers to floats.

All of the vectors (the parent N-sized and each of the 100-sized) must be freed by the user.

Parameters

fp	A string (char *) of the filepath to the binary dataset.
dimensions	The dimension of each point.

Returns

Vector* An N-sized vector with vectors for each element.

5.1.2.9 readBinGraph()

```
Graph * cppdescent::readBinGraph (
            const char * fp,
            int dimensions,
            DistanceFunc distance )
```

Reads a graph from a binary file. To work correctly, the file needs to be first created from the 'writeBinGraph' function.

Parameters

fp	The filepath.
dimensions	The dimensions of the datapoints.
distance	A function to compute the distance between the vertices.

Returns

Graph*

5.1.2.10 recall()

```
float cppdescent::recall (
            Graph * bfGraph,
            Graph * nnGraph,
            int N,
            int K )
```

Returns the recall of the graph computed by NN-Descent, compared to the brute force graph.

Parameters

bfGraph	
nnGraph	
N	
K	

Returns

float

5.1.2.11 writeBinGraph()

```
void cppdescent::writeBinGraph (
            const char * fp,
            Graph * graph,
            int K )
```

Writes a computed graph to a binary file.

The structure of the binary file that will be created is the following:

1. <uint32_t> N: the number of the vertices
2. N ∗ 100 floats: each vertex of 100 dimensions
3. N ∗ K ∗ 2 int: pairs of ints that describe the edges between the vertices. Each int represents the index of the vertex in the order above. This will be handy when we reconstruct the graph using the getAt() function of the vector.

Parameters

fp	The filepath to the created file.
K	
graph	

Chapter 6

Class Documentation

6.1 Graph Class Reference

Public Member Functions

- **Graph** (CompareFunc compare, DestroyFunc destroy, DestroyFunc vecDestroy=nullptr)
- int **getSize** ()
- void insertVertex (Pointer vertex)

  Insert a vertex to the graph (if it doesn't already exist).
- List * getVertices ()

  The returned list needs to be deleted, but the list's elements are pointers the the vector's elements, so the list shouldn't have a `destroyValue`.
- void **removeVertex** (Pointer vertex)
- void **insertEdge** (Pointer vertex1, Pointer vertex2, float weight)
- void **removeEdge** (Pointer vertex1, Pointer vertex2)
- float **getWeight** (Pointer vertex1, Pointer vertex2)
- List * **getAdjacent** (Pointer vertex)
- PQueue * **getAdjacentPQ** (Pointer vertex)
- List * **getReverseAdjacent** (Pointer vertex)
- PQueue * **getReverseAdjacentPQ** (Pointer vertex)
- List * **getGeneralNeighbors** (Pointer vertex)
- PQueue * **getGeneralNeighborsPQ** (Pointer vertex)
- bool **isNeighbor** (Pointer v1, Pointer v2)
- void **setHashFunction** (HashFunc hash)
- CompareFunc **getCompare** ()
- DestroyFunc **getDestroy** ()
- HashFunc **getHash** ()
- Vector * **getVec** ()
- Map * **getMap** ()

6.1.1 Member Function Documentation

6.1.1.1 getVertices()

```
List * Graph::getVertices ( )
```

The returned list needs to be deleted, but the list's elements are pointers the the vector's elements, so the list shouldn't have a `destroyValue`.

Returns

List*

6.1.1.2 insertVertex()

```
void Graph::insertVertex (
            Pointer vertex )
```

Insert a vertex to the graph (if it doesn't already exist).

Parameters

vertex	A Pointer to the vertex to be unserted.

The documentation for this class was generated from the following files:

- ADTGraph.hpp
- ADTGraph.cpp

6.2 GraphVertexPair Class Reference

Public Member Functions

- **GraphVertexPair** (Graph ∗owner, Pointer vertex1, Pointer vertex2)
- Pointer **getVertex1** ()
- Pointer **getVertex2** ()
- Graph ∗ **getOwner** ()

The documentation for this class was generated from the following file:

- ADTGraph.hpp

6.3 List Class Reference

ADT List.

```
#include <ADTList.hpp>
```

Public Member Functions

- List (DestroyFunc value=nullptr)

  Construct a new List object.
- ∼List ()

  Destroy the List object.
- void insertNext (ListNode ∗node, Pointer value)

  Adds a node after the given node.
- void removeNext (ListNode ∗node)

  Removes the node after the given node.
- Pointer find (Pointer value, CompareFunc compare)

Finds the first value equal to the value parameter.

- DestroyFunc setDestroyValue (DestroyFunc value)

  Set the Destroy Value object to a new value.
- int getSize ()

  Get the size of the List.
- int increaseSize ()

  Increase the list size.
- int decreaseSize ()

  Decrease the list size.
- ListNode * getHead ()

  Getter for the head of the List.
- ListNode * getTail ()

  Getter for the tail of the List.
- ListNode * next (ListNode *node)

  Returns the node after the given one.
- Pointer nodeValue (ListNode *node)

  Returns the value of the node.
- ListNode * findNode (Pointer value, CompareFunc compare)

  Finds the first node that has value equal to the value parameter.
- int mergeLists (List *list)

  Concatenation of 2 lists (Result this->list).

6.3.1 Detailed Description

ADT List.

An Abstract Data Type List with linear access to data, and addition and removal to/from arbitrary positions.

6.3.2 Constructor & Destructor Documentation

6.3.2.1 List()

```
List::List (
            DestroyFunc value = nullptr )
```

Construct a new List object.

Parameters

destroyValue	If destroyValue != nullptr, the destroyValue function will be called each time a node is removed.

head is a virtual node that is always present in the list.

Parameters

destroyValue	The destroy function for the values of the list. Defaults to nullptr.

6.3.2.2 ~List()

```
List::~List ( )
```

Destroy the List object.

If the destroyValue function is not nullptr, it is called for every value.

6.3.3 Member Function Documentation

6.3.3.1 decreaseSize()

```
int List::decreaseSize ( )
```

Decrease the list size.

Returns

int The updated size.

6.3.3.2 find()

```
Pointer List::find (
            Pointer value,
            CompareFunc compare )
```

Finds the first value equal to the value parameter.

Find the first value equal to the value parameter.

Parameters

value	The value based on which the search is done.
compare	A pointer to the function that will compare the node values.

Returns

Pointer A pointer to the resulting node.

Parameters

value	The value to search for.
compare	A pointer to the compare function.

Returns

Pointer A pointer to found value.

6.3.3.3 findNode()

```
ListNode * List::findNode (
            Pointer value,
            CompareFunc compare )
```

Finds the first node that has value equal to the value parameter.

Find the first node with value equal to the value parameter.

Parameters

value	The value to search.
compare	The compare function to use.

Returns

ListNode* A pointer to the resulting node.

Parameters

value	The value to search for.
compare	A pointer to the compare function.

Returns

ListNode* A pointer to the found node.

6.3.3.4 getHead()

```
ListNode * List::getHead ( )
```

Getter for the head of the List.

Get the first element of the List.

Returns

ListNode* The head.

Returns

ListNode* Pointer to the first element.

6.3.3.5 getSize()

```
int List::getSize ( )
```

Get the size of the List.

Get the size of the list.

The size is updated upon entry/removal of nodes, so the retrieval of the size is O(1).

Returns

int

Returns

int The size.

6.3.3.6 getTail()

```
ListNode * List::getTail ( )
```

Getter for the tail of the List.

Get the last element of the list.

Returns

ListNode* The tail.

If the list is empty, the virtual head is the tail, so we return nullptr.

Returns

ListNode* Pointer to the last element.

6.3.3.7 increaseSize()

```
int List::increaseSize ( )
```

Increase the list size.

Returns

int The updated size.

6.3.3.8 insertNext()

```
void List::insertNext (
            ListNode * node,
            Pointer value )
```

Adds a node *after* the given node.

Insert a new element after the node.

Parameters

node	The node that the new node will be placed after.
value	The value of the new node.

Parameters

node	The node after which the element is inserted.
value	The value of the new element.

6.3.3.9 mergeLists()

```
int List::mergeLists (
            List * list )
```

Concatenation of 2 lists (Result this->list).

Concatenation of 2 lists (Result: list1->list2).

Parameters

list	List to be merged with this.

Returns

int Returns 0 in case of success, -1 in any other case.

Parameters

list	List to be merged with this.

Returns

int Returns 0 in case of success, -1 in any other case.

6.3.3.10 next()

```
ListNode * List::next (
            ListNode * node )
```

Returns the node after the given one.

Get the next node.

Parameters

node	

Returns

ListNode* A pointer to the next node.

Parameters

node	

Returns

ListNode*

6.3.3.11 nodeValue()

```
Pointer List::nodeValue (
            ListNode * node )
```

Returns the value of the node.

Get the node's value.

Parameters

node	

Returns

Pointer A generic pointer to the value.

Parameters

node	

Returns

Pointer

6.3.3.12 removeNext()

```
void List::removeNext (
            ListNode * node )
```

Removes the node *after* the given node.

Removes the node after the given node.

Parameters

node	The node before the node to be removed.

Parameters

node	The node after which the element is removed.

6.3.3.13 setDestroyValue()

```
DestroyFunc List::setDestroyValue (
            DestroyFunc value )
```

Set the Destroy Value object to a new value.

Set the destroy value function.

Parameters

value	A pointer to the new destroy funciton.

Returns

DestroyFunc A pointer to the old destroy function.

Parameters

value	The new destroy function.

Returns

DestroyFunc The old destroy function.

The documentation for this class was generated from the following files:

- ADTList.hpp
- ADTList.cpp

6.4 ListNode Class Reference

Node of a list.

`#include <ADTList.hpp>`

Public Member Functions

- ListNode ()
  Construct a new List Node object that is empty.
- ListNode (Pointer value)
  Construct a new List Node object with value 'value'.
- void setNext (ListNode *next)
  Setter for the next node.
- ListNode * getNext ()
  Getter for the next node.
- Pointer getValue ()
  Getter for the value of the node.

6.4.1 Detailed Description

Node of a list.

A list node is represented by its value. It also holds a pointer to the next node in the list.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 ListNode() [1/2]

```
ListNode::ListNode ( )  [inline]
```

Construct a new List Node object that is empty.

6.4.2.2 ListNode() [2/2]

```
ListNode::ListNode (
            Pointer value )  [inline]
```

Construct a new List Node object with value 'value'.

Parameters

value	a generic pointer to the value of the node.

6.4.3 Member Function Documentation

6.4.3.1 getNext()

```
ListNode * ListNode::getNext ( )
```

Getter for the next node.

Get the next node of the list.

Returns

ListNode* The pointer to the next node.

Returns

ListNode* The next node.

6.4.3.2 getValue()

```
Pointer ListNode::getValue ( )
```

Getter for the value of the node.

Get the value of the current node.

Returns

Pointer A generic pointer to the value.

Returns

Pointer The value of the current node.

6.4.3.3 setNext()

```
void ListNode::setNext (
            ListNode * next )
```

Setter for the next node.

Set the given node as the next one.

Parameters

next	The pointer to the next node.

Parameters

next	The node to be next.

The documentation for this class was generated from the following files:

- ADTList.hpp
- ADTList.cpp

6.5 Map Class Reference

Public Member Functions

- Map (CompareFunc compare, DestroyFunc destroyKey, DestroyFunc destroyValue)

  Construct a new Map:: Map object.
- ~Map ()

  Destroy the Map:: Map object.
- void insert (Pointer key, Pointer value)

Add a new pair (key --> value) in the map.

- bool remove (Pointer key)

  Remove from the map the pair (key --> value) that corresponds to the given key.
- Pointer **find** (Pointer key)
- void rehash ()

  Extend the hash table in case the load factor exceeds the predefined threshold.
- void **setCapacity** (int capacity)
- void **setDestroyKey** (DestroyFunc destroyKey)
- void **setDestroyValue** (DestroyFunc destroyValue)
- void **setHashFunction** (HashFunc hash)
- MapNode ** **getArray** ()
- int **getCapacity** ()
- int **getSize** ()
- int **getDeleted** ()
- DestroyFunc **getDestroyKey** ()
- DestroyFunc **getDestroyValue** ()
- HashFunc **getHashFunction** ()
- MapNode * getFirst ()
- MapNode * getNext (MapNode *node)
- MapNode * findNode (Pointer key)

6.5.1 Constructor & Destructor Documentation

6.5.1.1 Map()

```
Map::Map (
            CompareFunc compare,
            DestroyFunc destroyKey,
            DestroyFunc destroyValue )
```

Construct a new Map:: Map object.

Parameters

compare	A compare function to keep the map sorted
destroyKey	A destroy function for the key of every node
destroyValue	A destroy function fro the balue of every node

6.5.1.2 ~Map()

```
Map::~Map ( )
```

Destroy the Map:: Map object.

6.5.2 Member Function Documentation

6.5.2.1 findNode()

```
MapNode * Map::findNode (
            Pointer key )
```

Parameters

key	

Returns

MapNode*

6.5.2.2 getFirst()

```
MapNode * Map::getFirst ( )
```

Returns

MapNode*

6.5.2.3 getNext()

```
MapNode * Map::getNext (
            MapNode * node )
```

Parameters

node	

Returns

MapNode*

6.5.2.4 insert()

```
void Map::insert (
            Pointer key,
            Pointer value )
```

Add a new pair (key --> value) in the map.

Parameters

key	
value	

6.5.2.5 rehash()

```
void Map::rehash ( )
```

Extend the hash table in case the load factor exceeds the predefined threshold.

Parameters

map	The map whose hash table needs to be rehashed

6.5.2.6 remove()

```
bool Map::remove (
            Pointer key )
```

Remove from the map the pair (key --> value) that corresponds to the given key.

Returns

true he node with the given key was removed successfully

false he node with the given key was not removed

The documentation for this class was generated from the following files:

- ADTMap.hpp
- ADTMap.cpp

6.6 MapNode Class Reference

Public Member Functions

- **MapNode** (Pointer key, Pointer value, State state)
- void **setKey** (Pointer key)
- void **setValue** (Pointer value)
- void **setState** (State state)
- Pointer **getKey** ()
- Pointer **getValue** ()
- State **getState** ()

The documentation for this class was generated from the following file:

- ADTMap.hpp

6.7 PQueue Class Reference

ADT Priority Queue.

```
#include <ADTPQueue.hpp>
```

Public Member Functions

- PQueue (CompareFunc compare, DestroyFunc destroyValue, Vector ∗values)

  Construct a new PQueue object.
- ∼PQueue ()

  Destroy the PQueue object.
- int getSize ()

  Get the size of the priority queue.
- Pointer getMax ()

  Get the max element of the queue.
- void insert (Pointer value)

  Insert a new element to the queue.
- void removeMax ()

  Remove the max of the queue.
- DestroyFunc setDestroyValue (DestroyFunc destroyValue)

  Set the Destroy Value object.
- Pointer **nodeValue** (int nodeId)
- void **nodeSwap** (int nodeId1, int nodeId2)
- void **bubbleUp** (int nodeId)
- void **bubbleDown** (int nodeId)
- void **naiveHeapify** (Vector ∗values)

6.7.1 Detailed Description

ADT Priority Queue.

6.7.2 Constructor & Destructor Documentation

6.7.2.1 PQueue()

```
PQueue::PQueue (
            CompareFunc compare,
            DestroyFunc destroyValue,
            Vector * values )
```

Construct a new PQueue object.

Uses the `compare` function to compare its elements.

If `destroyValue` != nullptr, the `destroyValue` function is called upon removal of an element.

If `values` != nullptr, then the priority queue will be initialized with the values of the vector `values`.

Parameters

compare	
destroyValue	
values	

6.7.2.2 ~PQueue()

```
PQueue::~PQueue ( )
```

Destroy the PQueue object.

6.7.3 Member Function Documentation

6.7.3.1 getMax()

```
Pointer PQueue::getMax ( )
```

Get the max element of the queue.

Returns

Pointer A generic pointer to the max element.

6.7.3.2 getSize()

```
int PQueue::getSize ( )
```

Get the size of the priority queue.

Returns

int

6.7.3.3 insert()

```
void PQueue::insert (
            Pointer value )
```

Insert a new element to the queue.

Parameters

value	The value of the new element to be inserted.

6.7.3.4 removeMax()

```
void PQueue::removeMax ( )
```

Remove the max of the queue.

6.7.3.5 setDestroyValue()

```
DestroyFunc PQueue::setDestroyValue (
            DestroyFunc destroyValue )
```

Set the Destroy Value object.

Parameters

destroyValue	

Returns

DestroyFunc

The documentation for this class was generated from the following files:

- ADTPQueue.hpp
- ADTPQueue.cpp

6.8 Vector Class Reference

ADT Vector.

```
#include <ADTVector.hpp>
```

Public Member Functions

- Vector (int size, DestroyFunc destroyValue)
  Construct a new Vector object.
- ~Vector ()
  Destroy the Vector object.
- int getSize ()
  Get the current size of the Vector.
- void insertLast (Pointer value)
  Inserts a new element at the end of the vector.
- int removeLast ()
  Removes the last element of the vector.
- Pointer getAt (int pos)
  Get the value at the specified position of the vector.
- int setAt (int pos, Pointer value)
  Set the value at the specified position of the vector.
- Pointer find (Pointer value, CompareFunc compare)
  Find the first element with value equal to value.
- int findPos (Pointer value, CompareFunc compare)
  Find the first element with value equal to value and return its pos.
- DestroyFunc setDestroyValue (DestroyFunc destroyValue)
  Set the Destroy Value.

- vectorNode * first ()

  Get the first node of the vector.
- vectorNode * last ()

  Get the last node of the vector.
- vectorNode * next (vectorNode *node)

  Get the next node of the vector, after the one given.
- vectorNode * previous (vectorNode *node)

  Get the previous node of the vector, before the one given.
- Pointer nodeValue (vectorNode *node)

  Get the value of the node.
- vectorNode * findNode (Pointer value, CompareFunc compare)

  Find the first node with value equal to value.

6.8.1 Detailed Description

ADT Vector.

An ADT Vector using a dynamic array for smart resizing.

6.8.2 Constructor & Destructor Documentation

6.8.2.1 Vector()

```
Vector::Vector (
            int size,
            DestroyFunc destroyValue )
```

Construct a new Vector object.

The newly created Vector will be of size size and its elements will be initialized as nullptr.

If destroyValue is not nullptr, it will be called on each element when it is removed from the Vector.

Parameters

size	The initial size of the Vector.
destroyValue	The function to be called on each element when it is removed from the Vector.

6.8.2.2 ~Vector()

```
Vector::~Vector ( )
```

Destroy the Vector object.

Deletes all allocated memory of the Vector.

6.8.3 Member Function Documentation

6.8.3.1 find()

```
Pointer Vector::find (
            Pointer value,
            CompareFunc compare )
```

Find the first element with value equal to value.

Parameters

value	The value to look for.
compare	The function to be used for comparison.

Returns

Pointer A pointer to the found value.

6.8.3.2 findNode()

```
vectorNode * Vector::findNode (
            Pointer value,
            CompareFunc compare )
```

Find the first node with value equal to value.

Parameters

value	The value to look for.
compare	The function to be used for comparison.

Returns

vectorNode The resulting node.

6.8.3.3 findPos()

```
int Vector::findPos (
            Pointer value,
            CompareFunc compare )
```

Find the first element with value equal to value and return its pos.

Parameters

value	The value to look for.
compare	The function to be used for comparison.

Returns

int The position of the element.

6.8.3.4 first()

```
vectorNode * Vector::first ( )
```

Get the first node of the vector.

Returns

vectorNode

6.8.3.5 getAt()

```
Pointer Vector::getAt (
            int pos )
```

Get the value at the specified position of the vector.

Parameters

pos	The position to look for.

Returns

Pointer A pointer to the value at the specified position.

6.8.3.6 getSize()

```
int Vector::getSize ( )
```

Get the current size of the Vector.

Returns

int The size of the Vector.

6.8.3.7 insertLast()

```
void Vector::insertLast (
            Pointer value )
```

Inserts a new element at the end of the vector.

Parameters

value	The value of the new element.

If we are at the last element, we create a new array double the size.

6.8.3.8 last()

```
vectorNode * Vector::last ( )
```

Get the last node of the vector.

Returns

vectorNode

6.8.3.9 next()

```
vectorNode * Vector::next (
            vectorNode * node )
```

Get the next node of the vector, after the one given.

Parameters

node	The node to get the next of.

Returns

vectorNode

6.8.3.10 nodeValue()

```
Pointer Vector::nodeValue (
            vectorNode * node )
```

Get the value of the node.

Parameters

node	

Returns

Pointer

6.8.3.11 previous()

```
vectorNode * Vector::previous (
            vectorNode * node )
```

Get the previous node of the vector, before the one given.

Parameters

node	The node to get the previous of.

Returns

vectorNode

6.8.3.12 removeLast()

```
int Vector::removeLast ( )
```

Removes the last element of the vector.

Returns

int -1 if the function fails, else 0.

6.8.3.13 setAt()

```
int Vector::setAt (
            int pos,
            Pointer value )
```

Set the value at the specified position of the vector.

Parameters

pos	The position to edit.
value	The new value.

Returns

int -1 if the function fails, else 0.

6.8.3.14 setDestroyValue()

```
DestroyFunc Vector::setDestroyValue (
            DestroyFunc destroyValue )
```

Set the Destroy Value.

Parameters

destroyValue	The new destroy value function.

Returns

DestroyFunc The old destroy value function.

The documentation for this class was generated from the following files:

- ADTVector.hpp
- ADTVector.cpp

6.9 vectorNode Class Reference

Class for the vector Node object.

`#include <ADTVector.hpp>`

Public Member Functions

- vectorNode ()
  Construct a new vector Node object.
- **vectorNode** (Pointer value)
- Pointer getValue () const
  Get the Value object.
- void setValue (Pointer value)
  Set the Value object.

6.9.1 Detailed Description

Class for the vector Node object.

A vector node is described only its value.

6.9.2 Constructor & Destructor Documentation

6.9.2.1 vectorNode()

`vectorNode::vectorNode ( )  [inline]`

Construct a new vector Node object.

Parameters

value	A Pointer to the value.

6.9.3 Member Function Documentation

6.9.3.1 getValue()

```
Pointer vectorNode::getValue ( ) const  [inline]
```

Get the Value object.

Returns

Pointer

6.9.3.2 setValue()

```
void vectorNode::setValue (
            Pointer value )  [inline]
```

Set the Value object.

Parameters

value	

The documentation for this class was generated from the following file:

- ADTVector.hpp

Chapter 7

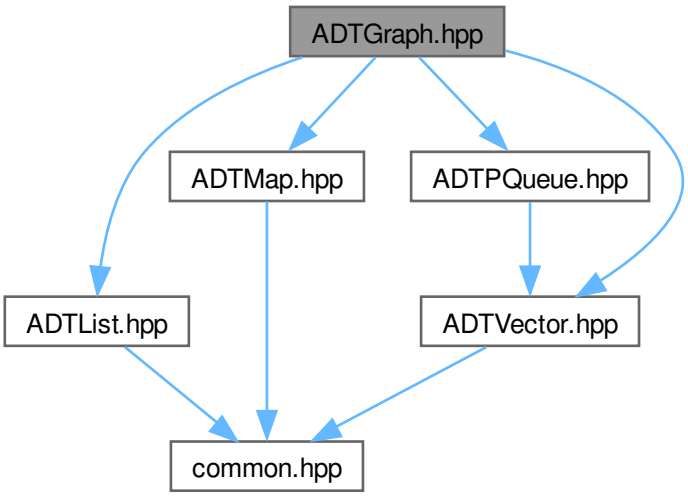
File Documentation

7.1 ADTGraph.hpp File Reference

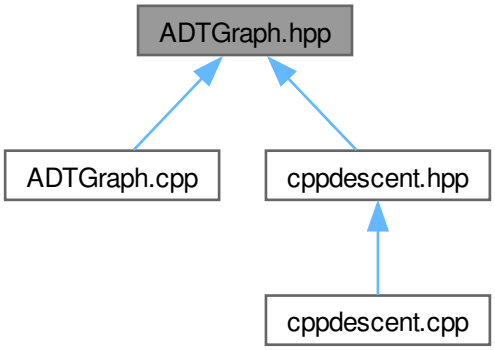
Abstract undirected graph with weighted edges.

```
#include "ADTList.hpp"
#include "ADTMap.hpp"
#include "ADTPQueue.hpp"
#include "ADTVector.hpp"
```

Include dependency graph for ADTGraph.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class Graph
- class GraphVertexPair

7.1.1 Detailed Description

Abstract undirected graph with weighted edges.

Author

Phaedon Seitanidis

Version

0.1

Date

2023-11-01

Copyright

Copyright (c) 2023

7.2 ADTGraph.hpp

Go to the documentation of this file.

```
00001
00012 #pragma once
00013
00014 #include "ADTList.hpp"
00015 #include "ADTMap.hpp"
00016 #include "ADTPQueue.hpp"
00017 #include "ADTVector.hpp"
00018
00019 class Graph {
00020  private:
00021   Vector* vec;
00022   Map* map;
00023   int size;
00024   CompareFunc compare;
00025   DestroyFunc destroy;
00026   HashFunc hash;
00027
00028  public:
00029   Graph(CompareFunc compare,
00030         DestroyFunc destroy,
00031         DestroyFunc vecDestroy = nullptr);
00032   ~Graph();
00033   int getSize();
00034   void insertVertex(Pointer vertex);
00035   List* getVertices();
00036   void removeVertex(Pointer vertex);
00037   void insertEdge(Pointer vertex1, Pointer vertex2, float weight);
00038   void removeEdge(Pointer vertex1, Pointer vertex2);
00039   float getWeight(Pointer vertex1, Pointer vertex2);
00040   List* getAdjacent(Pointer vertex);
00041   PQueue* getAdjacentPQ(Pointer vertex);
00042   List* getReverseAdjacent(Pointer vertex);
00043   PQueue* getReverseAdjacentPQ(Pointer vertex);
00044   List* getGeneralNeighbors(Pointer vertex);
00045   PQueue* getGeneralNeighborsPQ(Pointer vertex);
00046   bool isNeighbor(Pointer v1, Pointer v2);
00047   // Map* shortestPathLengths();
00048   void setHashFunction(HashFunc hash);
00049   CompareFunc getCompare() { return this->compare; };
00050   DestroyFunc getDestroy() { return this->destroy; };
00051   HashFunc getHash() { return this->hash; };
00052   Vector* getVec() { return this->vec; };
00053   Map* getMap() { return this->map; };
00054 };
00055
00056 class GraphVertexPair {
00057  public:
00058   GraphVertexPair(Graph* owner, Pointer vertex1, Pointer vertex2)
00059       : vertex1(vertex1), vertex2(vertex2), owner(owner){};
00060   Pointer getVertex1() { return this->vertex1; };
00061   Pointer getVertex2() { return this->vertex2; };
00062   Graph* getOwner() { return this->owner; };
00063
00064  private:
00065   Pointer vertex1;
00066   Pointer vertex2;
00067   Graph* owner;
00068 };
```

7.3 ADTList.hpp

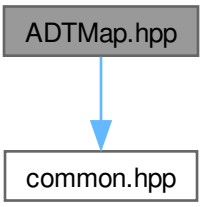
```
00001
00011 #pragma once
00012 #include "common.hpp"
00013
00014 #define LIST_BOF (ListNode*)0
00015 #define LIST_EOF (ListNode*)0
00016
00024 class ListNode {
00025  public:
00030   ListNode() : next(nullptr), value(nullptr){};
00036   ListNode(Pointer value) : next(nullptr), value(value){};
00042   void setNext(ListNode* next);
00048   ListNode* getNext();
00054   Pointer getValue();
00055
00056  private:
00057   ListNode* next;
```

```
00058   Pointer value;
00059 };
00060
00068 class List {
00069  public:
00076   List(DestroyFunc value = nullptr);
00081   ~List();
00088   void insertNext(ListNode* node, Pointer value);
00094   void removeNext(ListNode* node);
00102   Pointer find(Pointer value, CompareFunc compare);
00109   DestroyFunc setDestroyValue(DestroyFunc value);
00118   int getSize();
00119   int increaseSize();
00120   int decreaseSize();
00126   ListNode* getHead();
00132   ListNode* getTail();
00139   ListNode* next(ListNode* node);
00146   Pointer nodeValue(ListNode* node);
00154   ListNode* findNode(Pointer value, CompareFunc compare);
00161   int mergeLists(List* list);
00162
00163  private:
00164   DestroyFunc destroyValue;
00165   int size;
00166   ListNode* head;
00167   ListNode* tail;
00168 };
```

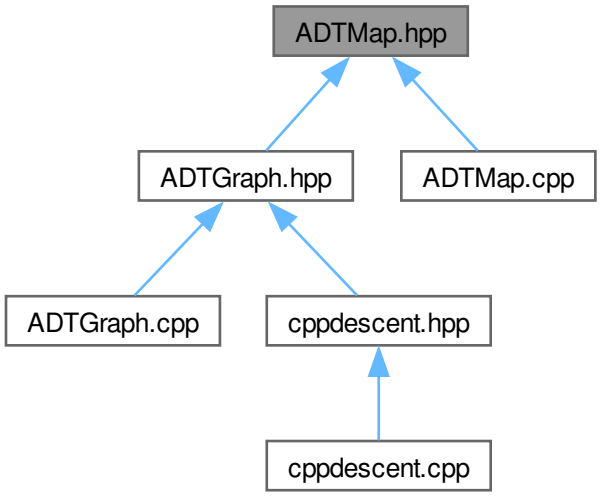
7.4 ADTMap.hpp File Reference

`#include ¨common.hpp¨`

Include dependency graph for ADTMap.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class MapNode
- class Map

Macros

- #define **MAX_LOAD_FACTOR** 0.5
- #define **MAP_EOF** (MapNode∗)0

Enumerations

- enum **State** { **EMPTY** , **OCCUPIED** , **DELETED** }

7.4.1 Detailed Description

Author

Phaedon Seitanidis

Version

0.1

Date

2023-11-01

Copyright

Copyright (c) 2023

7.5 ADTMap.hpp

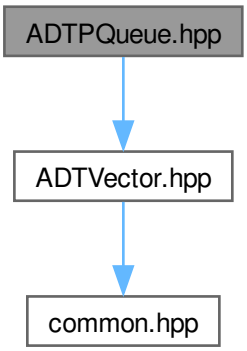
Go to the documentation of this file.

```
00001
00011 #pragma once
00012
00013 #include "common.hpp"
00014
00015 #define MAX_LOAD_FACTOR 0.5
00016 #define MAP_EOF (MapNode*)0
00017
00018 typedef enum { EMPTY, OCCUPIED, DELETED } State;
00019
00020 class MapNode {
00021  public:
00022   MapNode() : key(nullptr), value(nullptr), state(EMPTY){};
00023   MapNode(Pointer key, Pointer value, State state)
00024       : key(key), value(value), state(state){};
00025   ~MapNode(){};
00026   void setKey(Pointer key) { this->key = key; };
00027   void setValue(Pointer value) { this->value = value; };
00028   void setState(State state) { this->state = state; };
00029   Pointer getKey() { return this->key; };
00030   Pointer getValue() { return this->value; };
00031   State getState() { return this->state; };
00032
00033  private:
00034   Pointer key;
00035   Pointer value;
00036   State state;
00037 };
00038
00039 class Map {
00040  public:
00041   Map();
00042   Map(CompareFunc compare, DestroyFunc destroyKey, DestroyFunc destroyValue);
00043   ~Map();
00044   void insert(Pointer key, Pointer value);
00045   bool remove(Pointer key);
00046   Pointer find(Pointer key);
00047   void rehash();
00048   void setCapacity(int capacity) { this->capacity = capacity; };
00049   void setDestroyKey(DestroyFunc destroyKey) { this->destroyKey = destroyKey; };
00050   void setDestroyValue(DestroyFunc destroyValue) {
00051     this->destroyValue = destroyValue;
00052   };
00053   void setHashFunction(HashFunc hash) { this->hash = hash; };
00054   MapNode** getArray() { return this->array; };
00055   int getCapacity() { return this->capacity; };
00056   int getSize() { return this->size; };
00057   int getDeleted() { return this->deleted; };
00058   DestroyFunc getDestroyKey() { return this->destroyKey; };
00059   DestroyFunc getDestroyValue() { return this->destroyValue; };
00060   HashFunc getHashFunction() { return this->hash; };
00061   MapNode* getFirst();
00062   MapNode* getNext(MapNode* node);
00063   MapNode* findNode(Pointer key);
00064
00065  private:
00066   MapNode** array;
00067   CompareFunc compare;
00068   DestroyFunc destroyKey;
00069   DestroyFunc destroyValue;
00070   HashFunc hash;
00071   int capacity;
00072   int size;
00073   int deleted;
00074 };
```

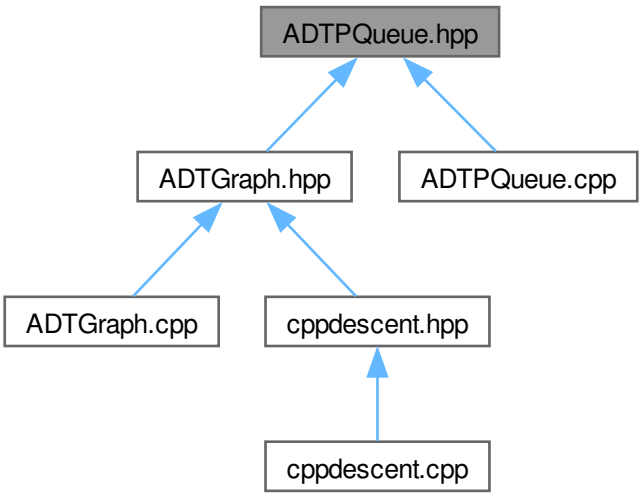
7.6 ADTPQueue.hpp File Reference

```
#include ¨ADTVector.hpp¨
```

Include dependency graph for ADTPQueue.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class PQueue

  ADT Priority Queue.

7.6.1 Detailed Description

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-30

Copyright

7.7 ADTPQueue.hpp

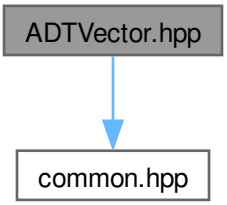
Go to the documentation of this file.

```
00001
00011 #pragma once
00012
00013 #include "ADTVector.hpp"
00014
00019 class PQueue {
00020  public:
00036   PQueue(CompareFunc compare, DestroyFunc destroyValue, Vector* values);
00041   ~PQueue();
00047   int getSize();
00053   Pointer getMax();
00059   void insert(Pointer value);
00064   void removeMax();
00071   DestroyFunc setDestroyValue(DestroyFunc destroyValue);
00072
00073   // Helper functions
00074   // These are used because the node IDs are 1-based, where as the
00075   // vector is 0-based.
00076   Pointer nodeValue(int nodeId);
00077   void nodeSwap(int nodeId1, int nodeId2);
00078   void bubbleUp(int nodeId);
00079   void bubbleDown(int nodeId);
00080   void naiveHeapify(Vector* values);
00081
00082  private:
00083   Vector* vector;
00084   CompareFunc compare;
00085   DestroyFunc destroyValue;
00086 };
```

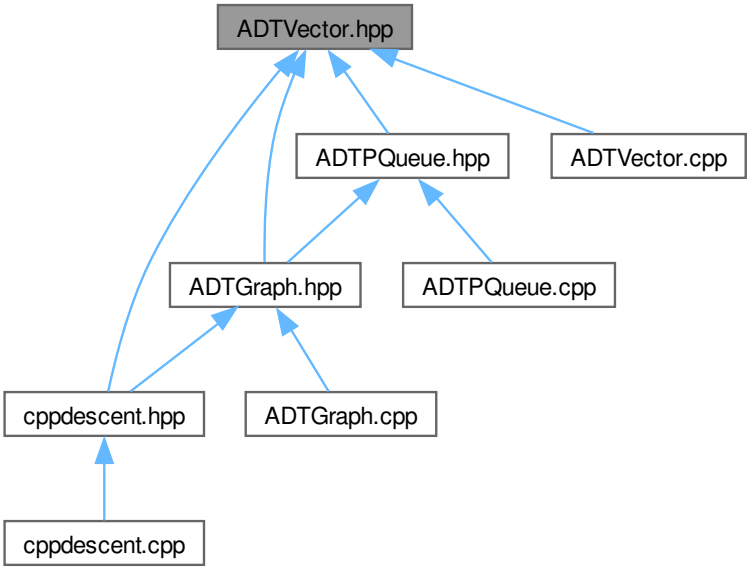
7.8 ADTVector.hpp File Reference

ADTVector implementation using a dynamic array.

`#include ¨common.hpp¨`
Include dependency graph for ADTVector.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class vectorNode
  Class for the vector Node object.
- class Vector
  ADT Vector.

Macros

- #define **VECTOR_MIN_CAPACITY** 10
- #define **VECTOR_BOF** (vectorNode∗)0
- #define **VECTOR_EOF** (vectorNode∗)0

7.8.1 Detailed Description

ADTVector implementation using a dynamic array.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-29

Copyright

7.9 ADTVector.hpp

Go to the documentation of this file.

```
00001
00011 #pragma once
00012
00013 #include "common.hpp"
00014
00015 #define VECTOR_MIN_CAPACITY 10
00016
00017 #define VECTOR_BOF (vectorNode*)0
00018 #define VECTOR_EOF (vectorNode*)0
00019
00026 class vectorNode {
00027  public:
00033   vectorNode(){};
00034   vectorNode(Pointer value) : value(value){};
00040   Pointer getValue() const { return value; };
00046   void setValue(Pointer value) { this->value = value; };
00047
00048  private:
00049   Pointer value;
00050 };
00051
00058 class Vector {
00059  public:
00073   Vector(int size, DestroyFunc destroyValue);
00080   ~Vector();
00086   int getSize();
00092   void insertLast(Pointer value);
00098   int removeLast();
00105   Pointer getAt(int pos);
00113   int setAt(int pos, Pointer value);
00121   Pointer find(Pointer value, CompareFunc compare);
00129   int findPos(Pointer value, CompareFunc compare);
00136   DestroyFunc setDestroyValue(DestroyFunc destroyValue);
00142   vectorNode* first();
00148   vectorNode* last();
00155   vectorNode* next(vectorNode* node);
00162   vectorNode* previous(vectorNode* node);
00169   Pointer nodeValue(vectorNode* node);
00177   vectorNode* findNode(Pointer value, CompareFunc compare);
00178
00179  private:
00180   vectorNode* array;
00181   int size;
00182   int capacity;
00183   DestroyFunc destroyValue;
00184 };
```

7.10 common.hpp

```
00001 #pragma once
00002
00003 typedef void* Pointer;
00004
00005 typedef int (*CompareFunc)(Pointer a, Pointer b);
00006
00007 typedef void (*DestroyFunc)(Pointer value);
00008
00009 typedef unsigned int (*HashFunc)(Pointer);
```

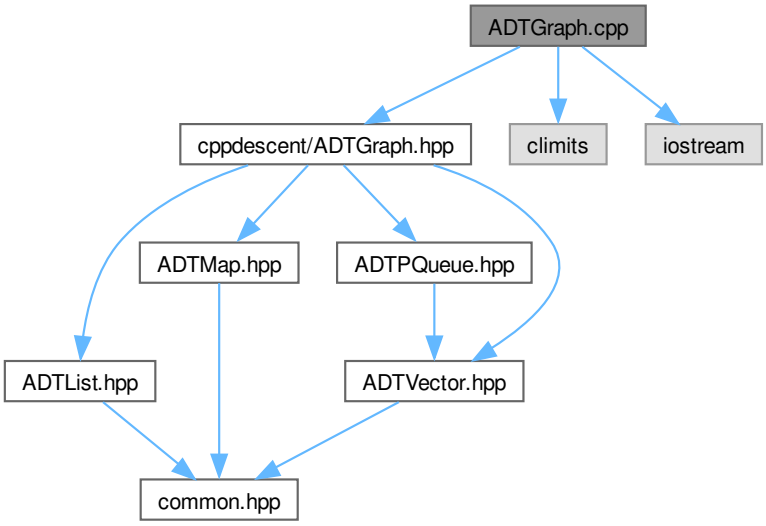
7.11 cppdescent.hpp

```
00001
00005 #include "ADTGraph.hpp"
00006 #include "ADTVector.hpp"
00007
00008 typedef float (*DistanceFunc)(Pointer a, Pointer b);
00009
00014 namespace cppdescent {
00015 void deleteFloat(Pointer value);
00016
00017 float* createFloat(float value);
00018
00019 float compareFloats(Pointer a, Pointer b);
00020
00021 int deleteDatapointVectors(Vector* vec);
00022
00023 int compareVertices(Pointer first, Pointer second);
00024
00025 // ================================ I/O ====================================
00040 Vector* readBinData(const char* fp, int dimensions);
00041
00060 void writeBinGraph(const char* fp, Graph* graph, int K);
00061
00071 Graph* readBinGraph(const char* fp, int dimensions, DistanceFunc distance);
00072
00073 // ============================ Helper Functions ============================
00084 float recall(Graph* bfGraph, Graph* nnGraph, int N, int K);
00085
00086 // ============================ Metric Functions ============================
00095 float euclideanDistance(Pointer a, Pointer b);
00104 float manhattanDistance(Pointer a, Pointer b);
00112 int compareEdgesEuclidean(Pointer first, Pointer second);
00120 int compareEdgesManhattan(Pointer first, Pointer second);
00121
00122 // ============================ KNN computation ============================
00138 Graph* KNNBruteForceGraph(Vector* data,
00139                           int K,
00140                           CompareFunc compare,
00141                           DistanceFunc distance);
00157 Graph* NNDescent_KNNGraph(Vector* data,
00158                           int K,
00159                           float delta,
00160                           DistanceFunc distance);
00172 List* NNDescent_Query(Graph* graph, int K, CompareFunc compare, Vector* query);
00173 };  // namespace cppdescent
```

7.12 ADTGraph.cpp File Reference

```
#include ¨cppdescent/ADTGraph.hpp¨
#include <climits>
#include <iostream>
```

Include dependency graph for ADTGraph.cpp:



Functions

- int * **createInt** (int value)
- float * **createFloat** (float value)
- int **compareVertexPair** (GraphVertexPair *pair1, GraphVertexPair *pair2)
- void **destroyVertexPair** (GraphVertexPair *pair)
- void **destroyValue** (Pointer value)
- int **compareEdgeWeights** (Pointer a, Pointer b)
- void **destroyEdgePair** (GraphVertexPair *pair)
- void **swap** (Pointer p, Pointer q)

7.12.1 Detailed Description

Author

Pheadon Seitanidis

Version

0.1

Date

2023-11-01

Copyright

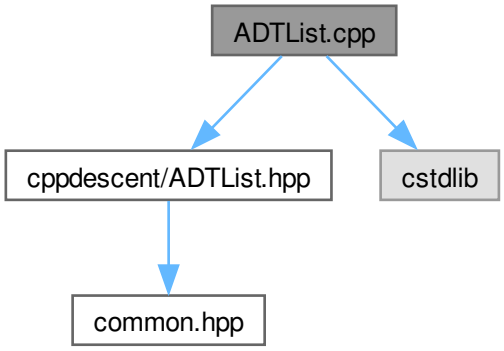
Copyright (c) 2023

7.13 ADTList.cpp File Reference

Implementation of an ADT linked list.

```
#include ¨cppdescent/ADTList.hpp¨
#include <cstdlib>
```

Include dependency graph for ADTList.cpp:



7.13.1 Detailed Description

Implementation of an ADT linked list.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-29

Copyright

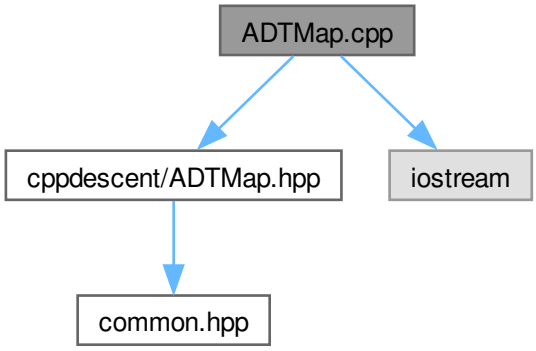
Copyright (c) 2023

7.14 ADTMap.cpp File Reference

Implementation of ADTMap.

```
#include ¨cppdescent/ADTMap.hpp¨
#include <iostream>
```

Include dependency graph for ADTMap.cpp:



Variables

- int prime_sizes [ ]

7.14.1 Detailed Description

Implementation of ADTMap.

Author

Phaedon Seitanidis

Version

0.1

Date

2023-10-29

Copyright

Copyright (c) 2023

7.14.2 Variable Documentation

7.14.2.1 prime_sizes

```
int prime_sizes[]
```

Initial value:

```
= {
    53,         97,         193,        389,        769,        1543,      3079,
    6151,       12289,      24593,      49157,      98317,      196613,    393241,
    786433,     1572869,    3145739,    6291469,    12582917,   25165843,  50331653,
    100663319,  201326611,  402653189,  805306457,  1610612741}
```

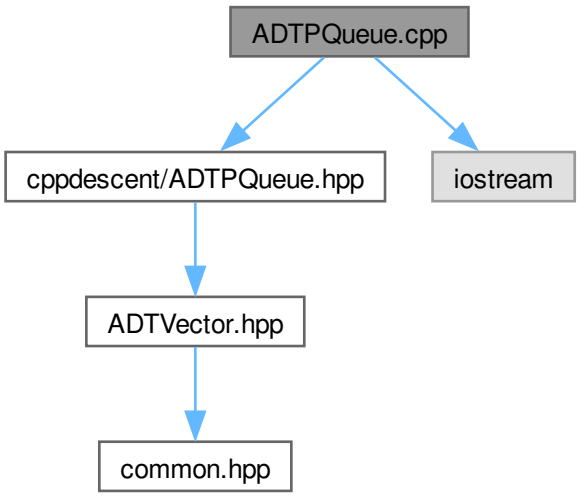
We want the capacity of the Hash Table to be a prime number, according to the theory. So the following list consists of prime numbers that are proven good capacity values for a Hash Table. When a rehash is needed, the capacity of the new table will be chosen from this table. If more than 1610612741 elements are needed, we double the capacity of the old table at rehash.

7.15 ADTPQueue.cpp File Reference

An ADT Priority Queue implemented using a heap.

```
#include ¨cppdescent/ADTPQueue.hpp¨
#include <iostream>
```

Include dependency graph for ADTPQueue.cpp:



7.15.1 Detailed Description

An ADT Priority Queue implemented using a heap.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-30

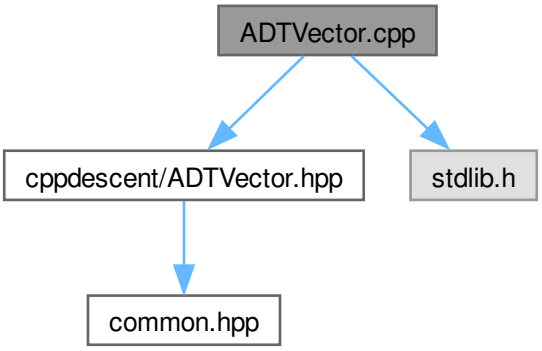
Copyright

7.16 ADTVector.cpp File Reference

Implementation of ADTVector using a dynamic array.

```
#include ¨cppdescent/ADTVector.hpp¨
#include ¨stdlib.h¨
```

Include dependency graph for ADTVector.cpp:



7.16.1 Detailed Description

Implementation of ADTVector using a dynamic array.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-30

Copyright

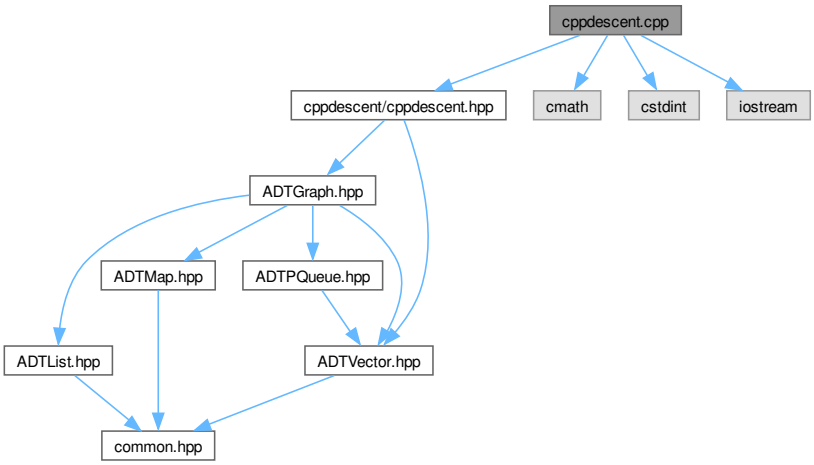
Copyright (c) 2023

7.17 cppdescent.cpp File Reference

Implementation of the cppdescent library.

```
#include ¨cppdescent/cppdescent.hpp¨
#include <cmath>
#include <cstdint>
#include <iostream>
```

Include dependency graph for cppdescent.cpp:



Functions

- void **swapEdges** (GraphVertexPair ∗first, GraphVertexPair ∗second)
- void **EdgesBubbleSort** (GraphVertexPair ∗∗edges, int size, CompareFunc compare)
- int **partition** (GraphVertexPair ∗∗edges, int low, int high, CompareFunc compare)
- void **EdgesQuickSort** (GraphVertexPair ∗∗edges, int low, int high, CompareFunc compare)
- bool **EdgesBinarySearch** (GraphVertexPair ∗∗edges, GraphVertexPair ∗edge, int low, int high, Compare↩ Func compare)
- void **destroyEdges** (GraphVertexPair ∗pair)
- uint **hashEdge** (Pointer value)
- void **deleteVectors** (Pointer vec)
- Graph ∗ sampleGraph (Vector ∗data, int K, CompareFunc compare, DistanceFunc distance)

  Creates a random graph, where each vertex has K random neighbors.
- int **updateNN** (Graph ∗graph, Pointer v, Pointer u2, float dist, DistanceFunc distance)
- List ∗ **NNDescent_Query** (Graph ∗graph, int K, CompareFunc compare, Vector ∗query)

7.17.1 Detailed Description

Implementation of the cppdescent library.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-27

Copyright

7.17.2 Function Documentation

7.17.2.1 sampleGraph()

```
Graph * sampleGraph (
            Vector * data,
            int K,
            CompareFunc compare,
            DistanceFunc distance )
```

Creates a random graph, where each vertex has K random neighbors.

The user is responsible for deallocating the graph.

Parameters

data	The Vector with all the points.
K	
compare	
distance	

Returns

Graph* The created graph.